

Gemini 3.8 Flash is now available. [Try it out](https://aistudio.google.com/prompts/new_chat?model=gemini-3.8-flash) (https://aistudio.google.com/prompts/new_chat?model=gemini-3.8-flash)

Nano Banana image generation



EXPERIMENT

Run in AI Studio



Nano Banana is the name for Gemini's native image generation capabilities. Gemini can generate and process images conversationally with text, images, video, or a combination. This lets you create, edit, and iterate on visuals with unprecedented control.

Model selection

Nano Banana refers to the following models available in the Gemini API:

- **Nano Banana 2.1 (Gemini Nano Banana 2.1** (/gemini-api/docs/models/gemini-nano-banana-2.1)) (**gemini-nano-banana-2.1**): An update to Nano Banana 2, serving as the primary high-efficiency workhorse model for image generation and conversational editing. It maintains Flash-level speed and cost efficiency while

delivering improved visual quality, text rendering, multi-turn consistency, and Google Search grounding across 1K, 2K, and 4K resolutions.

- **Nano Banana 2 Lite (Gemini 3.1 Flash Lite Image** (/gemini-api/docs/models/gemini-3.1-flash-lite-image)) (**gemini-3.1-flash-lite-image**): The fastest and cheapest Gemini image model, engineered for velocity and scale where speed and cost are the primary operational constraints. Not optimized for multiple reference inputs or multi-turn sequential editing.
- **Nano Banana 2 (Gemini 3.1 Flash Image** (/gemini-api/docs/models/gemini-3.1-flash-image)) (**gemini-3.1-flash-image**): The previous-generation high-efficiency workhorse model, balancing speed with 4K generation, world knowledge, and multi-reference image consistency. Recommended for all new projects to use Nano Banana 2.1.
- **Nano Banana Pro (Gemini 3 Pro Image** (/gemini-api/docs/models/gemini-3-pro-image)) (**gemini-3-pro-image**): The premium choice for the most complex visual tasks, offering the highest level of world knowledge, advanced localization, accurate brand consistency, and precision creative control.
- **Nano Banana (Gemini 2.5 Flash Image** (/gemini-api/docs/models/gemini-2.5-flash-image)) (**gemini-2.5-flash-image**): The legacy pioneer of the Nano Banana series. While it has been a reliable workhorse, recommended migration path is to transition to Nano Banana 2 Lite to experience enhanced quality, faster generation speeds, and lower API pricing.

All generated images include a [SynthID watermark](/responsible/docs/safeguards/synthid) (/responsible/docs/safeguards/synthid).

Image generation (text-to-image)

[Python](#) [JavaScript](#) (#javascript) [Java](#) (#java) [Go](#) (#go) [REST](#) (#rest)
(#python)

```
from google import genai
from PIL import Image
import base64

client = genai.Client()

interaction = client.interactions.create(
    model="gemini-nano-banana-2.1",
    input="Create a picture of a nano banana dish in a fancy restaurant with a Gemini theme",
)

with open("generated_image.png", "wb") as f:
    f.write(base64.b64decode(interaction.output_image.data))
```

You can retrieve generated image data by using the `interaction.output_image` property, which returns the last generated image block. For details on convenience properties, see the [Interactions overview](/gemini-api/docs/interactions-overview#convenience-properties) (/gemini-api/docs/interactions-overview#convenience-properties).

Image editing (text-and-image-to-image)

Reminder: Make sure you have the necessary rights to any images you upload. Don't generate content that infringe on others' rights, including videos or images that deceive, harass, or harm. Your use of this generative AI service is subject to our [Prohibited Use Policy](https://policies.google.com/terms/generative-ai/use-policy) (https://policies.google.com/terms/generative-ai/use-policy).

Provide an image and use text prompts to add, remove, or modify elements, change the style, or adjust the color grading.

The following example demonstrates uploading `base64` encoded images. For multiple images, larger payloads, and supported MIME types, check the [Image understanding](/gemini-api/docs/image-understanding) (/gemini-api/docs/image-understanding) page.

[Python](#) [JavaScript](#) (#javascript) [Java](#) (#java) [Go](#) (#go) [REST](#) (#rest)
(#python)

```
from google import genai
from PIL import Image
import base64

client = genai.Client()

with open("/path/to/cat_image.png", "rb") as f:
    image_bytes = f.read()

interaction = client.interactions.create(
    model="gemini-nano-banana-2.1",
    input=[
        {
            "type": "text",
            "text": "Create a picture of a nano banana dish in a fancy restaurant with a Gemini theme"
        },
        {
            "type": "image",
            "data": base64.b64encode(image_bytes).decode('utf-8'),
            "mime_type": "image/png"
        }
    ],
)

with open("generated_image.png", "wb") as f:
    f.write(base64.b64decode(interaction.output_image.data))
```

Multi-turn image editing

Keep generating and editing images conversationally. Multi-turn conversation is the recommended way to iterate on images. The following example shows a prompt to generate an infographic about photosynthesis.

[Python](#) [JavaScript](#) (#javascript) [Java](#) (#java) [Go](#) (#go) [REST](#) (#rest)
(#python)

```
from google import genai
import base64

client = genai.Client()

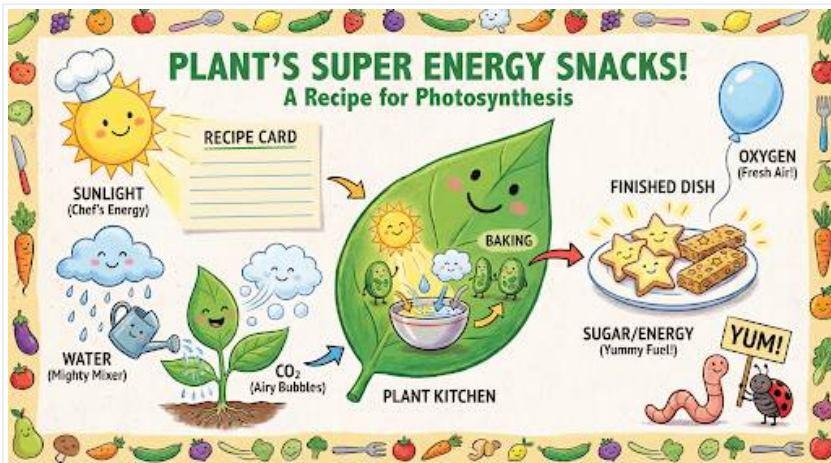
interaction = client.interactions.create(
```

```

model="gemini-nano-banana-2.1",
input="Create a vibrant infographic that explains photosynthesis as if it were a
recipe for a plant's favorite food. Show the \"ingredients\" (sunlight, water, CO2) and
the \"finished dish\" (sugar/energy). The style should be like a page from a colorful
kids' cookbook, suitable for a 4th grader.",
tools=[{"type": "google_search"}],
)

with open("photosynthesis.png", "wb") as f:
    f.write(base64.b64decode(interaction.output_image.data))

```



AI-generated infographic about photosynthesis

You can then use the `previous_interaction_id` to change the language on the graphic to Spanish.

PythonJavaScript (#javascript)Java (#java)Go (#go)REST (#rest)
(#python)

```

interaction_2 = client.interactions.create(
    model="gemini-nano-banana-2.1",
    input="Update this infographic to be in Spanish. Do not change any other elements of
the image.",
    previous_interaction_id=interaction.id,
    response_format={
        "type": "image",
        "mime_type": "image/jpeg",
        "aspect_ratio": "16:9",
        "image_size": "2K"
    },
)

generated_image = interaction_2.output_image
if generated_image:
    with open("photosynthesis_spanish.png", "wb") as f:
        f.write(base64.b64decode(generated_image.data))

```



AI-generated infographic of photosynthesis in Spanish

Features of Nano Banana

Nano Banana models offer state-of-the-art image generation and editing. Gemini Nano Banana 2.1 and Gemini 3.1 Flash Image are optimized for speed and high-volume use-cases, and Gemini 3 Pro Image is optimized for professional asset production. Designed to tackle the most challenging workflows through advanced reasoning, they excel at complex, multi-turn creation and modification tasks.

- **High-resolution output:** Built-in generation capabilities for 1K, 2K, and 4K visuals.
 - **Gemini 3.1 Flash Image** adds the smaller 512px (0.5K) resolution (not supported on Gemini Nano Banana 2.1).
 - **Gemini 3.1 Flash Lite Image** only supports 1K resolution.
- **Advanced text rendering:** Capable of generating legible, stylized text for infographics, menus, diagrams, and marketing assets.
- **Grounding with Google Search:** The model can use Google Search as a tool to verify facts and generate imagery based on real-time data (e.g., current weather maps, stock charts, recent events).
 - **Not supported by Gemini 3.1 Flash Lite Image model.**
 - **Gemini Nano Banana 2.1 and Gemini 3.1 Flash Image** add the integration of Google Image Search Grounding alongside Web Search.
- **Thinking mode:** The model utilizes a "thinking" process to reason through complex prompts. It generates interim "thought images" (visible in the backend but not charged) to refine the composition before producing the final high-quality output.
- **Up to 14 reference images:** Mix up to 14 reference images to produce the final image.
- **New aspect ratios:** Gemini 3.1 Flash Lite Image adds 1:1, 3:2, 2:3, 3:4, 4:3, 4:5, 5:4, 9:16, 16:9, 21:9 [aspect ratios](#) (#aspect-ratios-and-image-size).

Use up to 14 reference images

Nano Banana models allow you to mix up to 14 reference images. These 14 images can include the following:

Gemini 3.1 Flash Lite Image	Gemini Nano Banana 2.1 & Gemini 3.1 Flash Image	Gemini 3 Pro Image
Up to 14 images of objects with high-fidelity to include in the final image	Up to 10 images of objects with high-fidelity to include in the final image	Up to 6 images of objects with high-fidelity to include in the final image
N/A	Up to 4 images of characters to maintain character consistency	Up to 5 images of characters to maintain character consistency
N/A	N/A	Up to 3 images to be used as style references

Python (#python) JavaScript (#javascript) Java (#java) Go (#go) REST (#rest)

```
from google import genai
from google.genai import types
from PIL import Image
import base64

prompt = "An office group photo of these people, they are making funny faces."

client = genai.Client()

interaction = client.interactions.create(
    model="gemini-nano-banana-2.1",
    input=[
        {
            "type": "text",
            "text": prompt,
        },
        {
            "type": "image",
            "data": base64.b64encode(image_bytes).decode('utf-8'),
            "mime_type": "image/png"
        },
        {
            "type": "image",
            "data": base64.b64encode(image_bytes).decode('utf-8'),
            "mime_type": "image/png"
        },
        {
            "type": "image",
            "data": base64.b64encode(image_bytes).decode('utf-8'),
            "mime_type": "image/png"
        },
        {
            "type": "image",
            "data": base64.b64encode(image_bytes).decode('utf-8'),
            "mime_type": "image/png"
        },
    ],
)
```

```

    ],
    response_format={
        "type": "image",
        "aspect_ratio": "5:4",
        "image_size": "2K"
    },
)

with open("office.png", "wb") as f:
    f.write(base64.b64decode(interaction.output_image.data))

```



AI-generated office group photo

Grounding with Google Search

Use the [Google Search tool](/gemini-api/docs/google-search) (/gemini-api/docs/google-search) to generate images based on real-time information, such as weather forecasts, stock charts, or recent events.

Note that when using Grounding with Google Search with image generation, image-based search results are not passed to the generation model and are excluded from the response (see [Grounding with Google Image Search](#) (#image-search))

[Python](#) [JavaScript](#) (#javascript) [Java](#) (#java) [Go](#) (#go) [REST](#) (#rest)
(#python)

```

from google import genai
from google.genai import types
import base64
prompt = "Visualize the current weather forecast for the next 5 days in San Francisco as a clean, modern weather chart. Add a visual on what I should wear each day"

client = genai.Client()

interaction = client.interactions.create(

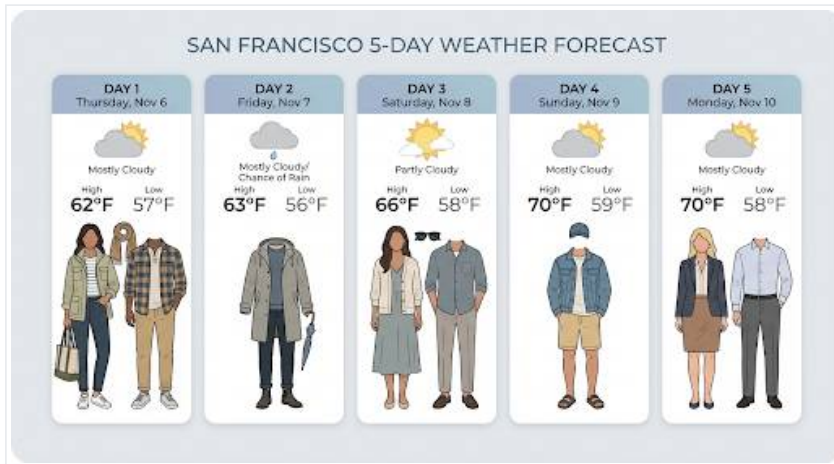
```

```

model="gemini-nano-banana-2.1",
input=prompt,
tools=[{"type": "google_search"}],
response_format={
    "type": "image",
    "mime_type": "image/jpeg",
    "aspect_ratio": "16:9"
},
)

with open("weather.png", "wb") as f:
    f.write(base64.b64decode(interaction.output_image.data))

```



AI-generated five day weather chart for San Francisco

The response includes `google_search_call` and `google_search_result` steps, along with inline `url_citation` annotations on the text step:

- **google_search_result:** Contains `search_suggestions`, an HTML snippet for rendering search suggestions in your UI.
- **url_citation annotations:** Inline citations on the text step linking parts of the response to their web sources.

Grounding with Google Search for images (Nano Banana 2.1 and 3.1 Flash)

Grounding with Google Image Search allows models to use web images retrieved via Google Image Search as visual context for image generation. Image Search is a new search type within the existing Grounding with Google Search tool, functioning alongside standard [Web Search](#) (`#use-with-grounding`).

To enable Image Search, configure the `google_search` tool in your API request and specify `image_search` within the `search_types` array. Image Search can be used independently or together with Web Search.

[Python](#) [JavaScript](#) (`#javascript`) [Java](#) (`#java`) [Go](#) (`#go`) [REST](#) (`#rest`)
(`#python`)

```

from google import genai

client = genai.Client()

interaction = client.interactions.create(
    model="gemini-nano-banana-2.1",

```

```

input="A detailed painting of a Timareta butterfly resting on a flower",
tools=[{
  "type": "google_search",
  "search_types": ["web_search", "image_search"]
}]
)

```

Display requirements

When you use Image Search within Grounding with Google Search, you must display the `search_suggestions` from the `google_search_result` step. Full usage requirements are detailed in the [Terms of Service \(/gemini-api/terms#grounding-with-google-search\)](https://ai.google.dev/gemini-api/terms#grounding-with-google-search).

Response

For grounded responses using image search, the API returns inline citations and attribution metadata as part of the response steps:

- **url_citation_annotations:** Inline citations on the text content block within `model_output`, linking the generated content to its source.
- **google_search_result:** Contains `search_suggestions`, an HTML snippet for rendering search suggestions in your UI.

Video-to-image generation (Nano Banana 2.1, 3.1 Flash, and 3.1 Flash Lite)

Video-to-image generation allows you to generate new images using a video's context as a multimodal reference. This is useful for creating high-quality video thumbnails, cinematic posters, summary infographics, or new artwork inspired by a video scene.

During generation, the model analyzes the video frames in context to extract visual themes and key events, then uses them alongside your text prompt to synthesize the output image.

You can pass public [YouTube URLs](https://ai.google.dev/gemini-api/docs/video-understanding#youtube) (<https://ai.google.dev/gemini-api/docs/video-understanding#youtube>) directly in your API request or upload local video files using the [Files API](https://ai.google.dev/gemini-api/docs/files) ([/gemini-api/docs/files](https://ai.google.dev/gemini-api/docs/files)).

[Python](#) [JavaScript](#) (#javascript) [Java](#) (#java) [Go](#) (#go) [REST](#) (#rest)
(#python)

```

from google import genai
from google.genai import types
import base64

client = genai.Client()

interaction = client.interactions.create(
    model="gemini-nano-banana-2.1",
    input=[
        {
            "type": "video",
            "uri": "https://www.youtube.com/watch?v=UTdfxFy0QTI",
            "mime_type": "video/mp4"
        },
        {"type": "text", "text": "Generate a poster image that captures the key themes of this video."}
    ]
)

```

```

],
response_format={"type": "image", "aspect_ratio": "16:9"}
)

# Save the generated image part
for step in interaction.steps:
    if step.type == "model_output":
        for content_block in step.content:
            if content_block.type == "text":
                print(content_block.text)
            elif content_block.type == "image":
                with open("video_poster.png", "wb") as f:
                    f.write(base64.b64decode(content_block.data))
                print("Image saved as video_poster.png")

```

BUILDING WITH NANO BANANA (GEMINI 2.5 FLASH IMAGE)

GETTING STARTED

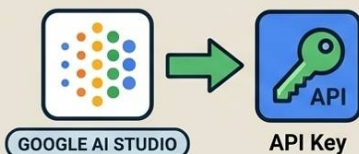


IMAGE CREATION



IMAGE EDITING



MULTIPLE IMAGES



PHOTO RESTORATION



CONVERSATIONAL EDITING



AI-generated infographic from a youtube video

Generate images up to 4K resolution

Gemini 3 image models generate 1K images by default but can also output 2K, 4K, and 512px (05.K) (Gemini 3.1 Flash Image only) images. To generate higher resolution assets, specify the `image_size` in the `response_format`.

You must use an uppercase 'K' (e.g. 512px (05.K), 1K, 2K, 4K). Lowercase parameters (e.g., 1k) will be rejected.

PythonJavaScript (#javascript)Java (#java)Go (#go)REST (#rest)
(#python)

```

from google import genai
from google.genai import types
import base64

```

```

prompt = "Da Vinci style anatomical sketch of a dissected Monarch butterfly. Detailed drawings of the head, wings, and legs on textured parchment with notes in English."

```

```

client = genai.Client()

interaction = client.interactions.create(
    model="gemini-nano-banana-2.1",
    input=prompt,
    response_format={
        "type": "image",
        "mime_type": "image/jpeg",
        "aspect_ratio": "1:1",
        "image_size": "1K"
    },
)

print(interaction.output_text)

with open("butterfly.png", "wb") as f:
    f.write(base64.b64decode(interaction.output_image.data))

```

The following is an example image generated from this prompt:



AI-generated Da Vinci style anatomical sketch of a dissected Monarch butterfly.

Thinking process

Gemini 3 image models are thinking models that use a reasoning process ("Thinking") for complex prompts. This feature is enabled by default and cannot be disabled in the API. To learn more about the thinking process, see the [Gemini Thinking](#) (/gemini-api/docs/thinking) guide.

The model generates up to two interim images to test composition and logic. The last image within Thinking is also the final rendered image.

You can check the thoughts that lead to the final image being produced.

PythonJavaScript (#javascript)**Java** (#java)**Go** (#go)
(#python)

```

for step in interaction.steps:
    if step.type == "thought":
        for content_block in step.summary:
            if content_block.type == "text":
                print(content_block.text)

```

```

elif content_block.type == "image":
    image = Image.open(io.BytesIO(base64.b64decode(content_block.data)))
    image.show()

```

Interleaved text and images

While standard image generation models only output images, some advanced Gemini 3 models (such as `gemini-3-pro-image`) can generate interleaved content—like stories or instructional guides containing both text blocks and illustrations inside the same response.

Because the output is complex and interleaved, convenience properties like `.output_image` or `.output_text` will not capture the full sequence. To access and save interleaved content, you must manually iterate over `steps`:

[Python](#)
[JavaScript](#) (#javascript)
[Java](#) (#java)
[Go](#) (#go)
[\(#python\)](#)

```

interaction = client.interactions.create(
    model="gemini-3-pro-image",
    input="Write the story of the lifecycle of a monarch butterfly, interleave
illustrations",
)

image_counter = 1
for step in interaction.steps:
    if step.type == "model_output":
        for content_block in step.content:
            if content_block.type == "text":
                print(content_block.text)
            elif content_block.type == "image":
                filename = f"butterfly_lifecycle_{image_counter}.png"
                with open(filename, "wb") as f:
                    f.write(base64.b64decode(content_block.data))
                print(f"\n[Saved illustration: {filename}]\n")
                image_counter += 1

```

Controlling thinking levels

With Gemini Nano Banana 2.1, Gemini 3.1 Flash Image, and Gemini 3.1 Flash Lite Image, you can control the amount of thinking the model uses to balance quality and latency:

Model	Supported thinking levels	Default
Gemini Nano Banana 2.1	minimal, medium, high	medium
Gemini 3.1 Flash Image	minimal, high	minimal
Gemini 3.1 Flash Lite Image	minimal, high	minimal

[Python](#)
[JavaScript](#) (#javascript)
[Java](#) (#java)
[Go](#) (#go)
[REST](#) (#rest)
[\(#python\)](#)

```

from google import genai
from PIL import Image
import base64
import io

client = genai.Client()

interaction = client.interactions.create(
    model="gemini-nano-banana-2.1",
    input="A futuristic city built inside a giant glass bottle floating in space",
    generation_config={"thinking_level": "high"},
)

print(interaction.output_text)

image = Image.open(io.BytesIO(base64.b64decode(interaction.output_image.data)))

image.show()

```

Note that thinking tokens are billed by default for thinking models, as the [thinking process](#) (`#thinking-process`) always happens by default whether you view the process or not.

Other image generation modes

Although Nano Banana image generation models are recommended for most use cases, you can also explore dedicated image generation models:

- **Imagen:** Google's legacy text-to-image models (shut down).
- **[Veo](#)** (`/gemini-api/docs/video`): Google's video generation model.

Generate images in batch

All of the image generation capabilities described on this page can also be run as batch jobs using the [Batch API](#) (`/gemini-api/docs/batch-api#image-generation`), which is ideal if you need to generate many images. You get higher rate limits in exchange for a turnaround of up to 24 hours.

Prompting guide and strategies

This section provides prompt examples and templates for common image generation and editing workflows. Each example includes a re-usable template and a sample prompt for the Interactions API.

Prompts for generating images

The following examples show how to use text prompts to generate various types of images.

1. Photorealistic scenes

Describe a scene in rich detail. The more specific you are, the more control you have over the results.

TemplatePrompt (#prompt)Python (#python)JavaScript (#javascript)Java (#java)Go (#go)REST (#rest)
(#template)

A photorealistic [type of shot] of a [subject description] in a [setting description]. [Description of the light]. Shot from a [camera angle] with a [lens type].

2. Stylized illustrations & stickers

Describe the artistic style, subject, and medium. Be specific about the visual detail (bold lines, colors, etc.) for consistent results.

TemplatePrompt (#prompt)Python (#python)JavaScript (#javascript)Java (#java)Go (#go)REST (#rest)
(#template)

A [style] of a [subject, with details about accessories or actions] doing [activity]. The design features [visual qualities, e.g., bold outlines, cel-shading, etc.] and [color/background preference].



A kawaii-style sticker of a happy red panda...

3. Accurate text in images

Gemini excels at rendering text. Be clear about the text, the font style (descriptively), and the overall design. Use Gemini 3 Pro Image for professional asset production.

Create a [image type] for [brand/concept] with the text "[text to render]" in a [font style]. The design should be [style description], with a [color scheme].



Create a modern, minimalist logo for a coffee shop called 'The Daily Grind'...

4. Product mockups & commercial photography

Perfect for creating clean, professional product shots for ecommerce, advertising, or branding.

TemplatePrompt (#prompt)Python (#python)JavaScript (#javascript)Java (#java)Go (#go)REST (#rest)
(#template)

A high-resolution, studio-lit product photograph of a [product description] on a [background surface/description]. The lighting is a [lighting setup, e.g., three-point softbox setup] to [lighting purpose]. The camera angle is a [angle type] to showcase [specific feature]. Ultra-realistic, with sharp focus on [key detail]. [Aspect ratio].



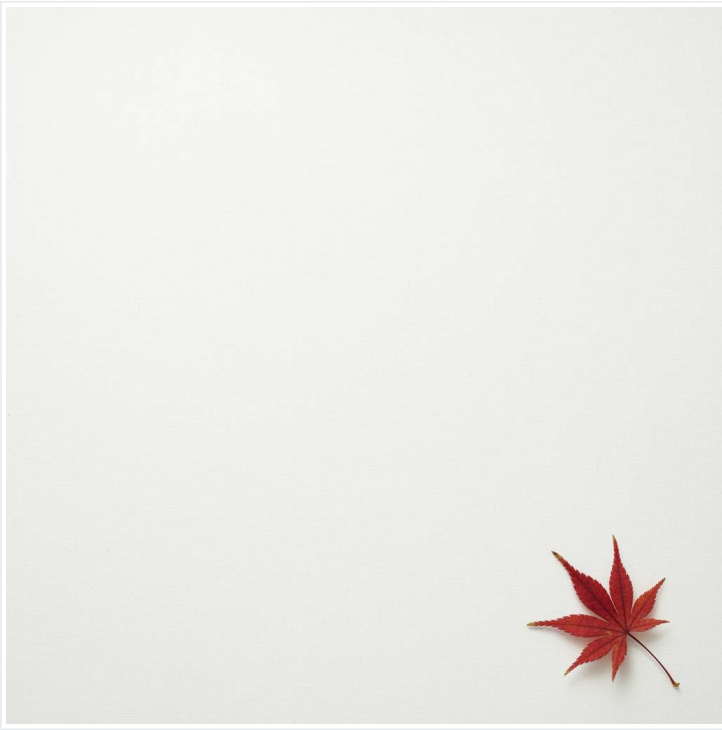
A high-resolution, studio-lit product photograph of a minimalist ceramic coffee mug...

5. Minimalist & negative space design

Excellent for creating backgrounds for websites, presentations, or marketing materials where text will be overlaid.

TemplatePrompt (#prompt)Python (#python)JavaScript (#javascript)Java (#java)Go (#go)REST (#rest)
(#template)

A minimalist composition featuring a single [subject] positioned in the [bottom-right/top-left/etc.] of the frame. The background is a vast, empty [color] canvas, creating significant negative space. Soft, subtle lighting. [Aspect ratio].



A minimalist composition featuring a single, delicate red maple leaf...

6. Sequential art (comic panel / storyboard)

Builds on character consistency and scene description to create panels for visual storytelling. For accuracy with text and storytelling ability, these prompts work best with Gemini 3 Pro and Gemini 3.1 Flash Image.

TemplatePrompt (#prompt)Python (#python)JavaScript (#javascript)Java (#java)Go (#go)REST (#rest)
(#template)

Make a 3 panel comic in a [style]. Put the character in a [type of scene].

Input



Input image

Output



Make a 3 panel comic in a gritty, noir art style...

7. Grounding with Google Search

Use Google Search to generate images based on recent or real-time information. This is useful for news, weather, and other time-sensitive topics.

[Prompt](#)[Python](#) (#python)[JavaScript](#) (#javascript)[Java](#) (#java)[Go](#) (#go)[REST](#) (#rest)
(#prompt)

Make a simple but stylish graphic of last night's Arsenal game in the Champion's League



AI-generated graphic of an Arsenal football score

Prompts for editing images

These examples show how to provide images alongside your text prompts for editing, composition, and style transfer.

1. Adding and removing elements

Provide an image and describe your change. The model will match the original image's style, lighting, and perspective.

[Template](#)[Prompt](#) (#prompt)[Python](#) (#python)[JavaScript](#) (#javascript)[Java](#) (#java)[Go](#) (#go)[REST](#) (#rest)
(#template)

Using the provided image of [subject], please [add/remove/modify] [element] to/from the scene. Ensure the change is [description of how the change should integrate].

Input	Output



A photorealistic picture of a fluffy ginger cat...



Using the provided image of my cat, please add a small, knitted wizard hat...

2. Inpainting (semantic masking)

Con conversationally define a "mask" to edit a specific part of an image while leaving the rest untouched.

Template Prompt (#prompt)Python (#python)JavaScript (#javascript)Java (#java)Go (#go)REST (#rest)
(#template)

Using the provided image, change only the [specific element] to [new element/description]. Keep everything else in the image exactly the same, preserving the original style, lighting, and composition.

Input

Output



A wide shot of a modern, well-lit living room...



Using the provided image of a living room, change only the blue sofa to be a vintage, brown leather chesterfield sofa...

3. Style transfer

Provide an image and ask the model to recreate its content in a different artistic style.

Template Prompt (#prompt)Python (#python)JavaScript (#javascript)Java (#java)Go (#go)REST (#rest)
(#template)

Transform the provided photograph of [subject] into the artistic style of [artist/art style]. Preserve the original composition but render it with [description of stylistic elements].

Input



A photorealistic, high-resolution photograph of a busy city street...

Output



Transform the provided photograph of a modern city street at night...

4. Advanced composition: combining multiple images

Provide multiple images as context to create a new, composite scene. This is perfect for product mockups or creative collages.

TemplatePrompt (#prompt)Python (#python)JavaScript (#javascript)Java (#java)Go (#go)REST (#rest) (#template)

Create a new image by combining the elements from the provided images. Take the [element from image 1] and place it with/on the [element from image 2]. The final image should be a [description of the final scene].

Input 1



Input 2



Output



A blue floral summer dress on a neutral background

Full-body shot of a woman with her hair in a bun...

A woman wearing a blue floral summer dress in an outdoor setting

5. High-fidelity detail preservation

To ensure critical details (like a face or logo) are preserved during an edit, describe them in great detail along with your edit request.

[TemplatePrompt](#) (#prompt)[Python](#) (#python)[JavaScript](#) (#javascript)[Java](#) (#java)[Go](#) (#go)[REST](#) (#rest)
(#template)

Using the provided images, place [element from image 2] onto [element from image 1]. Ensure that the features of [element from image 1] remain completely unchanged. The added element should [description of how the element should integrate].

Input 1

Input 2

Output



A professional headshot of a woman with brown hair and blue eyes...



Modern brand identifier with letters G and A



Take the first image of the woman with brown hair, blue eyes, and a neutral expression...

6. Bring something to life

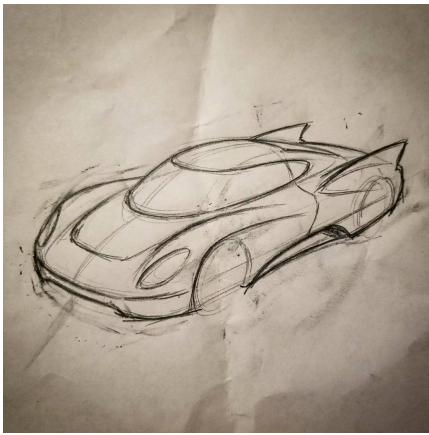
Upload a rough sketch or drawing and ask the model to refine it into a finished image.

[TemplatePrompt](#) (#prompt)[Python](#) (#python)[JavaScript](#) (#javascript)[Java](#) (#java)[Go](#) (#go)[REST](#) (#rest)
(#template)

Turn this rough [medium] sketch of a [subject] into a [style description] photo. Keep the [specific features] from the sketch but add [new details/materials].

Input

Output



Rough sketch of a car



Polished photo of a car

7. Character consistency: 360 view

You can generate 360-degree views of a character by iteratively prompting for different angles. For best results, include previously generated images in subsequent prompts to maintain consistency. For complex poses, include a reference image of the selected pose.

Template Prompt (#prompt) Python (#python)
(#template)

A studio portrait of [person] against [background], [looking forward/in profile looking right/etc.]

Input

Output 1

Output 2



Original image



Man in white glasses looking right



Man in white glasses looking forward

Best practices

To elevate your results from good to great, incorporate these professional strategies into your workflow.

- **Be hyper-specific:** The more detail you provide, the more control you have. Instead of "fantasy armor," describe it: "ornate elven plate armor, etched with silver leaf patterns, with a high collar and pauldrons"

shaped like falcon wings."

- **Provide context and intent:** Explain the *purpose* of the image. The model's understanding of context will influence the final output. For example, "Create a logo for a high-end, minimalist skincare brand" will yield better results than just "Create a logo."
- **Iterate and refine:** Don't expect a perfect image on the first try. Use the conversational nature of the model to make small changes. Follow up with prompts like, "That's great, but can you make the lighting a bit warmer?" or "Keep everything the same, but change the character's expression to be more serious."
- **Use step-by-step instructions:** For complex scenes with many elements, break your prompt into steps. "First, create a background of a serene, misty forest at dawn. Then, in the foreground, add a moss-covered ancient stone altar. Finally, place a single, glowing sword on top of the altar."
- **Use "semantic negative prompts":** Instead of saying "no cars," describe the intended scene positively: "an empty, deserted street with no signs of traffic."
- **Control the camera:** Use photographic and cinematic language to control the composition. Terms like `wide-angle shot`, `macro shot`, `low-angle perspective`.

Limitations

- For best performance, use the following languages: EN, ar-EG, de-DE, es-MX, fr-FR, hi-IN, id-ID, it-IT, ja-JP, ko-KR, pt-BR, ru-RU, ua-UA, vi-VN, zh-CN.
- Image generation does not support audio inputs. Video inputs are only supported for Gemini Nano Banana 2.1, Gemini 3.1 Flash Image, and Gemini 3.1 Flash Lite Image.
- The model won't always follow the exact number of image outputs that the user explicitly asks for.
- `gemini-2.5-flash-image` works best with up to 3 images as input, while `gemini-3-pro-image` supports 5 images with high fidelity, and up to 14 images in total. `gemini-nano-banana-2.1` and `gemini-3.1-flash-image` support character resemblance of up to 4 characters and the fidelity of up to 10 objects in a single workflow.
- When generating text for an image, Gemini works best if you first generate the text and then ask for an image with the text.
- `gemini-nano-banana-2.1` and `gemini-3.1-flash-image` Grounding with Google Search do not support using real-world images of people from web search at this time.
- All generated images include a [SynthID watermark](/responsible/docs/safeguards/synthid) (/responsible/docs/safeguards/synthid).

Optional configurations

You can optionally configure the output format, aspect ratio, and image size using the `response_format` parameter.

Output format

The model defaults to returning both text and image responses. You can configure the response to return only the generated images (omitting the conversational text) by specifying an image format in the `response_format`

parameter.

To request multiple modalities (for example, both text and the generated image), pass an array of format entries to `response_format` instead.

[Python](#) [JavaScript](#) (#javascript) [Java](#) (#java) [Go](#) (#go) [REST](#) (#rest)
(#python)

```
interaction = client.interactions.create(
    model="gemini-nano-banana-2.1",
    input="Write a short poem about a starry night and generate an image of it.",
    response_format=[
        {"type": "text"},
        {"type": "image"},
    ],
)
```

Aspect ratios and image size

By default, the model matches the output image size to that of your input image, or otherwise generates 1:1 squares. You can control the aspect ratio and the size of the output image using the `aspect_ratio` and `image_size` fields under `response_format` when `type` is set to `"image"`.

[Python](#) [JavaScript](#) (#javascript) [Java](#) (#java) [Go](#) (#go) [REST](#) (#rest)
(#python)

```
interaction = client.interactions.create(
    model="gemini-nano-banana-2.1",
    input=prompt,
    response_format={
        "type": "image",
        "aspect_ratio": "16:9",
        "image_size": "2K",
    },
)
```

The different ratios available and the size of the image generated are listed in the following tables:

Nano Banana 2.13.1 Flash Image... (#nano-banana-2.1)			3.1 Pro Image (#3.1-pro-image) Gemini 2.5 Flash Image...			
Aspect ratio	1K resolution	1K tokens	2K resolution	2K tokens	4K resolution	4K tokens
1:1	1024x1024	1120	2048x2048	1680	4096x4096	2520
1:4	512x2048	1120	1024x4096	1680	2048x8192	2520
1:8	384x3072	1120	768x6144	1680	1536x12288	2520
2:3	848x1264	1120	1696x2528	1680	3392x5056	2520

Aspect ratio	1K resolution	1K tokens	2K resolution	2K tokens	4K resolution	4K tokens
3:2	1264x848	1120	2528x1696	1680	5056x3392	2520
3:4	896x1200	1120	1792x2400	1680	3584x4800	2520
4:1	2048x512	1120	4096x1024	1680	8192x2048	2520
4:3	1200x896	1120	2400x1792	1680	4800x3584	2520
4:5	928x1152	1120	1856x2304	1680	3712x4608	2520
5:4	1152x928	1120	2304x1856	1680	4608x3712	2520
8:1	3072x384	1120	6144x768	1680	12288x1536	2520
9:16	768x1376	1120	1536x2752	1680	3072x5504	2520
16:9	1376x768	1120	2752x1536	1680	5504x3072	2520
21:9	1584x672	1120	3168x1344	1680	6336x2688	2520

When to use Imagen

Imagen is shut down and no longer available through the Gemini API. Use Nano Banana for image generation and editing.

What's next

- Check out the [Veo guide \(/gemini-api/docs/video\)](/gemini-api/docs/video) to learn how to generate videos with the Gemini API.
- To learn more about Gemini models, see [Gemini models \(/gemini-api/docs/models/gemini\)](/gemini-api/docs/models/gemini).

Except as otherwise noted, the content of this page is licensed under the [Creative Commons Attribution 4.0 License \(https://creativecommons.org/licenses/by/4.0/\)](https://creativecommons.org/licenses/by/4.0/), and code samples are licensed under the [Apache 2.0 License \(https://www.apache.org/licenses/LICENSE-2.0\)](https://www.apache.org/licenses/LICENSE-2.0). For details, see the [Google Developers Site Policies \(https://developers.google.com/site-policies\)](https://developers.google.com/site-policies). Java is a registered trademark of Oracle and/or its affiliates.

Last updated 2026-10-09 UTC.